

THE GARAGE

COMPLETE LEARNING GUIDE

Learn Web Development from Zero to Production

HTML · CSS · JavaScript · Python · Django · Docker · CI/CD · AWS

CONTENTS

- | | |
|----------------------------------|-------------------------------------|
| 01 What is Web Development? | 11 Authentication — Login System |
| 02 HTML — Structure of Web Pages | 12 The UPI Payment Flow |
| 03 CSS — Styling and Layout | 13 PDF Generation with ReportLab |
| 04 JavaScript — Interactivity | 14 Testing — All 23 Tests |
| 05 Python — The Language | 15 Docker — Containerization |
| 06 Django — The Framework | 16 Jenkins — CI/CD Pipeline |
| 07 Database Models | 17 Terraform — Cloud Infrastructure |
| 08 Forms — User Input | 18 Ansible — Deployment |
| 09 Views — Business Logic | 19 Industry Standards |
| 10 Templates — HTML Layer | 20 What to Learn Next |

CHAPTER 01

What is Web Development?

Understanding how the internet works before writing any code

CHAPTER 01

What is Web Development?

When you open a website like Google or Instagram on your browser, a lot of things happen behind the scenes in less than one second. Web development is the act of building those things. It is split into two main areas: **Frontend** and **Backend**.

Frontend — What the User Sees

The frontend is everything you see and interact with in your browser. The text, buttons, images, colors, animations — all of this is built with three technologies that every web browser understands:

- **HTML** — HyperText Markup Language. Defines the structure. Think of it as the skeleton of a webpage.
- **CSS** — Cascading Style Sheets. Defines the appearance. Colors, fonts, sizes, layouts.
- **JavaScript** — The programming language that runs in the browser. Makes things interactive — clicking buttons, animations, counting numbers.

Backend — What the Server Does

The backend is the code running on a server (a computer in a data center) that the user never directly sees. When you click 'Book your service', the backend saves your booking into a database and sends back a confirmation page. THE GARAGE's backend is built with:

- **Python** — The programming language used on the server.
- **Django** — A Python framework that handles web requests, database access, and HTML rendering.
- **SQLite** — A simple file-based database (used during development).

How a Browser Request Works

This is the most important concept to understand. Every time a browser loads a page, this exact sequence happens:

```
Step 1: You type http://localhost:8000/ in your browser
Step 2: Browser sends an HTTP request to the server:
        GET / HTTP/1.1
```

Host: localhost:8000

Step 3: Django's URL router matches "/" to the home() view function

Step 4: home() queries the database:

```
SELECT * FROM garage_service WHERE is_active = 1 LIMIT 3
```

Step 5: Django fills the home.html template with real data from database

Step 6: Server sends back an HTTP response:

```
HTTP/1.1 200 OK
```

```
Content-Type: text/html
```

```
<html>...</html>
```

Step 7: Browser receives the HTML and renders it on your screen

■ **NOTE:** HTTP stands for HyperText Transfer Protocol — the language browsers and servers speak to each other. GET means 'give me a page'. POST means 'accept data I am sending you'.

URL	What you type in the browser address bar: http://localhost:8000/book/
HTTP Request	Message browser sends to server asking for a page
HTTP Response	What the server sends back: HTML, JSON, a file, etc.
Status 200 OK	Request succeeded — page found and returned
Status 404	Page not found
Status 403	Forbidden — you don't have permission
Status 500	Server error — something went wrong in the code
GET	Retrieving data — clicking a link or typing a URL
POST	Sending data — submitting a form
localhost:8000	Your own computer running a development server on port 8000

CHAPTER 02

HTML

HyperText Markup Language — The Structure of Web Pages

CHAPTER 02

HTML — Structure of Web Pages

HTML is not a programming language — it is a **markup language**. You use it to describe the structure of a page using **tags**. Every HTML tag has an opening tag and a closing tag:

```
<!-- Opening tag      Content      Closing tag -->
<h1>                  welcome      </h1>

<!-- A paragraph -->
<p>This is a paragraph of text.</p>

<!-- A link -->
<a href="https://google.com">Click here</a>

<!-- An image -->


<!-- A button -->
<button type="button">Book Service</button>

<!-- A form – collects data from user -->
<form method="post" action="/book/">
  <input type="text" name="vehicle" placeholder="Your car model">
  <input type="date" name="date">
  <button type="submit">Submit</button>
</form>
```

The Complete HTML Page Structure

```
<!DOCTYPE html>          <!-- tells browser: this is HTML5 -->
<html lang="en">          <!-- root element of the whole page -->
  <head>                  <!-- metadata – not visible on page -->
    <meta charset="UTF-8"> <!-- character encoding (handles ■ ★ etc.) -->
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>THE GARAGE</title> <!-- browser tab title -->
    <link rel="stylesheet" href="/static/css/site.css"> <!-- load CSS file -->
  </head>

  <body>                  <!-- everything visible goes here -->
```

```
<header>
  <nav>
    <a href="/">Home</a>
    <a href="/services/">Services</a>
    <a href="/book/">Book</a>
  </nav>
</header>

<main>
  <h1>WE KEEP YOU ROLLING.</h1>
  <p>Thoughtful service. Straight answers.</p>
  <a href="/book/" class="button">Book your service</a>
</main>

<footer>
  <p>THE GARAGE — Est. 1998</p>
</footer>

<script src="/static/js/site.js"></script> <!-- load JS at end of body -->
</body>
</html>
```

Important HTML Tags Used in This Project

<section>	A distinct section of a page — hero, reviews, footer
<article>	A self-contained item — a service card, a review, a booking row
<div>	A generic container block — used for grouping and layout
	A generic inline container — used for styling part of text
<picture>	Container for responsive images with multiple sources
<source>	An image source inside , used to serve WebP first
	Display an image
<form>	Collects input from the user
<input>	Text box, date picker, checkbox, hidden field
<select>	Dropdown menu
<textarea>	Multi-line text input
<button>	Clickable button
<label>	Text label attached to a form input
class=	Attribute that links element to CSS style rules
id=	Unique identifier for one specific element
href=	The URL a link points to
action=	The URL a form submits data to
method=post	Form sends data via POST (hidden in request body, not URL)
{% csrf_token %}	Django security token inserted into every POST form
aria-label=	Describes element to screen readers — accessibility

fetchpriority=high

Tells browser to load this image first (LCP optimization)

CHAPTER 03

CSS

Cascading Style Sheets — Making the Page Look Good

CHAPTER 03

CSS — Styling and Layout

CSS is how you make a page beautiful. You write rules that say: *"find all elements with this class, and apply these visual properties to them."*

Basic CSS Syntax

```
/* selector { property: value; } */

/* Target all <h1> elements */
h1 {
  font-size: 24px;
  color: #e34935;      /* red */
  font-weight: bold;
}

/* Target elements with class="button" */
.button {
  background: #e34935;
  color: white;
  padding: 12px 20px;  /* top/bottom 12px, left/right 20px */
  border: none;
  cursor: pointer;
}

/* Target the element with id="hero" */
#hero {
  min-height: 600px;
  background: #111214;
}

/* Target .button when hovered */
.button:hover {
  background: #f05a44; /* slightly lighter on hover */
  transform: translateY(-2px); /* lift up 2px */
}
```

CSS Custom Properties (Design Tokens)

Instead of writing the same color code #e34935 everywhere, you define a **variable** once and use it everywhere. If the brand color changes, you change it in ONE place.

```
:root {
  /* Define variables on the root element */
  --ink:      #111214;
  --paper:    #f2f0eb;
  --red:      #e34935;
  --lime:     #c6d27f;
  --display:  "Barlow Condensed", sans-serif;
  --body:     "DM Sans", sans-serif;
}

/* Use with var() */
.button-red {
  background: var(--red);    /* uses the variable */
  color: var(--paper);
  font-family: var(--display);
}

h1 {
  color: var(--red);    /* same variable – change once, updates everywhere */
}
```

CSS Flexbox — Side-by-Side Layout

```
.hero-actions {
  display: flex;           /* make children sit side by side */
  align-items: center;     /* vertically center children */
  gap: 26px;              /* space between children */
}

/* Now these two elements sit side by side */
<div class="hero-actions">
  <a class="button button-red">Book service</a>    ← left
  <a class="text-link">Explore services</a>         ← right
</div>
```

CSS Grid — Complex Layouts

```
.service-grid {
  display: grid;
  grid-template-columns: repeat(3, minmax(0, 1fr));
  /* repeat(3, ...) = 3 columns
     1fr = each column gets equal fraction of available space
     minmax(0, 1fr) = column can shrink to 0 but grows equally */
  gap: 1px;
  background: rgba(255,255,255,0.14); /* shows as gap color */
}

/* Service cards become a 3-column grid automatically */
<div class="service-grid">
  <article class="service-card">Oil Change</article>
  <article class="service-card">Brake Service</article>
  <article class="service-card">Tyre Rotation</article>
</div>
```

CSS Animations — The Hero Car

```

/* Define the animation with @keyframes */
@keyframes road-car-drive {
  0% { opacity: 0.15; transform: translate3d(32vw, 13px, 0) scale(0.9); }
  /* 0% = start: car is far right, faded, small */
  62% { transform: translate3d(12vw, -5px, 0) scale(1.015); }
  /* 62% = moves past center (overshoot) */
  79% { transform: translate3d(-2vw, 4px, 0) skewY(0.5deg); }
  /* 79% = bounces back left a bit, slight lean */
  100% { opacity: 1; transform: translate3d(0, 0, 0) scale(1); }
  /* 100% = settles at final position */
}

/* Apply the animation to the car image */
.hero-road-car {
  animation: road-car-drive 2.75s cubic-bezier(.2,.68,.22,1) both;
}
/* 2.75s = animation takes 2.75 seconds
   cubic-bezier = custom easing curve (physics-based)
   both = apply start state before, and end state after animation */

/* Content fades in after car arrives */
.hero-sequence .hero-title {
  animation: hero-reveal 0.72s ease both;
  animation-delay: 3.3s; /* wait 3.3s after page loads */
}

```

Responsive Design — Mobile Friendly

```

/* Default styles: desktop (large screen) */
.service-grid {
  grid-template-columns: repeat(3, 1fr); /* 3 columns */
}

/* tablet — screen width 900px or less */
@media (max-width: 900px) {
  .service-grid {
    grid-template-columns: repeat(2, 1fr); /* 2 columns */
  }
}

/* mobile — screen width 680px or less */
@media (max-width: 680px) {
  .service-grid {
    grid-template-columns: 1fr; /* 1 column (full width) */
  }
  .hero-spec-bar {
    display: none; /* hide spec bar on small screens */
  }
}

/* Accessibility: user has asked OS to reduce motion */
@media (prefers-reduced-motion: reduce) {
  * {
    animation: none !important; /* disable all animations */
  }
  .hero-road-car {
    opacity: 1;
    transform: none; /* show car without animation */
  }
}

```

CHAPTER 04

JavaScript

Making Pages Interactive — Animations, Clicks, and Events

CHAPTER 04

JavaScript — Making Pages Interactive

JavaScript (JS) is the only programming language that runs directly inside the browser. It makes pages interactive — responding to clicks, animating elements, counting numbers, and updating content without reloading the page.

JavaScript Basics

```
// Variables
const name = "THE GARAGE";    // const = cannot be reassigned
let count = 0;                // let = can be reassigned

// Functions
function greet(person) {
  return "Hello, " + person;
}
const result = greet("Sujan"); // result = "Hello, Sujan"

// Arrow functions (shorter syntax)
const double = (n) => n * 2;

// Arrays
const services = ["Oil Change", "Brake Check", "Tyre Rotation"];
services.forEach((service) => console.log(service));

// Objects
const booking = {
  id: 42,
  customer: "Sujan",
  service: "Oil Change",
  isPaid: false,
};
console.log(booking.customer); // "Sujan"

// Selecting elements from HTML
const button = document.querySelector('.button-red');
const allCards = document.querySelectorAll('.service-card');

// Adding event listeners
```

```
button.addEventListener('click', () => {
  alert("Booking confirmed!");
});
```

The IntersectionObserver — Scroll Reveal

This is how service cards, review cards, and why-us cards fade in as you scroll down the page. Instead of constantly checking scroll position (which is slow), IntersectionObserver watches for when an element enters the visible area of the screen:

```
// Create an observer
const revealObserver = new IntersectionObserver((entries) => {
  entries.forEach((entry) => {
    if (entry.isIntersecting) {      // element is now in viewport
      entry.target.classList.add('is-visible'); // add CSS class
      revealObserver.unobserve(entry.target);    // stop watching (only once)
    }
  });
}, { threshold: 0.12 }); // fire when 12% of the element is visible

// Tell the observer which elements to watch
document.querySelectorAll('.service-card, .review, .why-card').forEach((item) => {
  item.classList.add('reveal');      // starts hidden (opacity:0)
  revealObserver.observe(item);      // start watching
});

/* CSS: .reveal starts invisible, .is-visible shows it */
/* .reveal { opacity: 0; transform: translateY(13px); transition: 0.5s; } */
/* .reveal.is-visible { opacity: 1; transform: translateY(0); } */
```

Count-Up Animation — The Trust Strip Numbers

The numbers 25+, 4.9★, and 12k+ animate from 0 when you scroll to them:

```
function animateCount(el) {
  const target = parseInt(el.dataset.target, 10); // read data-target="25" from HTML
  const duration = 1400; // animation lasts 1.4 seconds
  const start = performance.now(); // high-precision timer

  function tick(now) {
    const elapsed = Math.min(now - start, duration);
    const progress = elapsed / duration; // 0.0 → 1.0

    // Cubic ease-out: starts fast, slows near the end (natural motion)
    const eased = 1 - Math.pow(1 - progress, 3);

    el.textContent = Math.round(eased * target); // update number on screen
    if (elapsed < duration) {
      requestAnimationFrame(tick); // ask browser to call tick() next frame (60fps)
    } else {
      el.textContent = target; // set exact final value
    }
  }
  requestAnimationFrame(tick); // start animation
}

// Watch for the trust strip to enter viewport, then animate
const countObserver = new IntersectionObserver((entries) => {
  entries.forEach((entry) => {
    if (entry.isIntersecting) {
      animateCount(entry.target);
    }
  });
});
```

```
        countObserver.unobserve(entry.target);
    }
});
}, { threshold: 0.5 });

document.querySelectorAll('.count-up').forEach((el) => countObserver.observe(el));
```

CHAPTER 05

Python

The Programming Language Behind Django

CHAPTER 05

Python — The Language Behind Django

Python is one of the most popular programming languages in the world. It is used for web development (Django), data science (pandas), machine learning (TensorFlow), automation, and much more. It is famous for being easy to read and write.

```
# Variables – no type declaration needed
name = "THE GARAGE"
price = 2500.00
is_open = True

# String formatting with f-strings
booking_ref = f"TG-{42:06d}" # TG-000042
message = f"Your booking {booking_ref} is confirmed!"

# Lists and loops
services = ["Oil Change", "Brake Service", "Tyre Rotation"]
for service in services:
    print(service)

# Dictionaries (key-value pairs)
booking = {
    "customer": "Sujan",
    "service": "Oil Change",
    "is_paid": False,
}
print(booking["customer"]) # Sujan
print(booking.get("notes", "No notes")) # "No notes" if key doesn't exist

# Functions
def format_price(amount):
    return f"■{amount:,.2f}"

print(format_price(2500)) # ■2,500.00

# Classes – the foundation of Django models
class Booking:
    def __init__(self, customer, service, price):
        self.customer = customer
```

```

        self.service = service
        self.price = price
        self.is_paid = False # default value

    def mark_paid(self):
        self.is_paid = True
        return f"{self.customer}'s booking is now paid."

    @property
    def total_with_tax(self):
        return self.price * 1.18 # 18% GST

b = Booking("Sujan", "Oil Change", 2500)
print(b.total_with_tax) # 2950.0 (calculated, not stored)
print(b.mark_paid())

# List comprehensions (concise filtering)
all_bookings = [...] # imagine a list of Booking objects
upcoming = [b for b in all_bookings if b.is_upcoming and not b.is_paid]
# Same as:
# upcoming = []
# for b in all_bookings:
#     if b.is_upcoming and not b.is_paid:
#         upcoming.append(b)

# Exception handling
try:
    result = 100 / 0
except ZeroDivisionError as e:
    print(f"Error: {e}")
finally:
    print("This always runs")

```

Python Decorators — Used Heavily in Django

```

# A decorator is a function that WRAPS another function
# It runs extra code BEFORE or AFTER the wrapped function

def login_required(view_function):
    def wrapper(request):
        if not request.user.is_authenticated:
            return redirect('/accounts/login/') # stop here, redirect to login
        return view_function(request) # otherwise, run the real view
    return wrapper

# Apply decorator with @ syntax
@login_required
def dashboard(request):
    # This code only runs if user is logged in
    return render(request, 'dashboard.html', {...})

# Equivalent to:
# dashboard = login_required(dashboard)

```

CHAPTER 06

Django

The Web Framework — MVT Architecture

CHAPTER 06

Django — The Web Framework

Django is a high-level Python web framework that lets you build web applications quickly and cleanly. It follows the **MVT pattern** (Model–View–Template), which is Django's version of the famous MVC (Model–View–Controller) pattern.

The MVT Pattern Explained

Model (M)	Python class that represents a database table. Booking, Service, GarageLocation are all Models. Django converts them to SQL tables automatically.
View (V)	Python function that handles a web request and returns a response. Queries models, passes data to templates. views.py contains all 17 view functions.
Template (T)	HTML file with special Django tags like <code>{{ variable }}</code> and <code>{% if %}</code> . View passes data here, template renders it as final HTML.
URL Router	urls.py maps URL paths to view functions. <code>/</code> → <code>home()</code> , <code>/book/</code> → <code>book_service()</code> , etc.

How Django Processes a Request — Step by Step

User visits: `http://localhost:8000/account/bookings/42/invoice/`

Step 1: `the_garage/urls.py` receives the request
`include('garage.urls')` → passes to `garage/urls.py`

Step 2: `garage/urls.py` finds the match:
`path("account/bookings/<int:booking_id>/invoice/", views.invoice_detail)`
Extracts `booking_id = 42`

Step 3: `views.invoice_detail(request, booking_id=42)` runs:
`booking = get_object_or_404(`
 `request.user.bookings.select_related("service", "location"),`
 `pk=42`
`)`
SQL: `SELECT * FROM booking`
`JOIN service ON booking.service_id = service.id`
`JOIN garage_location ON booking.location_id = location.id`

```
# WHERE booking.customer_id = 7      (logged-in user)
# AND booking.id = 42
```

```
Step 4: return render(request, "garage/invoice.html", {
    "booking": booking,
    "invoice_total": booking.effective_price,
    "payment_form": UpiPaymentConfirmationForm(),
})
Django loads templates/garage/invoice.html
Fills in {{ booking.vehicle }}, {{ invoice_total }}, etc.
```

Step 5: Browser receives finished HTML and displays the invoice page

Settings.py — The Configuration File

```
# the_garage/settings.py

import os
from pathlib import Path

BASE_DIR = Path(__file__).resolve().parent.parent
# __file__ = this settings.py file
# .parent.parent = go up two directories = project root

# ■■■ SECRET KEY — Never expose this!
SECRET_KEY = os.getenv("DJANGO_SECRET_KEY", "dev-insecure-key")
# os.getenv() reads from environment variable
# If not set, falls back to the default (dev only)

# DEBUG mode — ALWAYS False in production
DEBUG = os.getenv("DJANGO_DEBUG", "True").lower() in ("true", "1", "yes")

# Database — SQLite for development
DATABASES = {
    "default": {
        "ENGINE": "django.db.backends.sqlite3",
        "NAME": BASE_DIR / "db.sqlite3", # db.sqlite3 in project root
    }
}

# For production PostgreSQL:
# "ENGINE": "django.db.backends.postgresql"
# "NAME": "garage_db", "USER": "postgres", "PASSWORD": "...", "HOST": "localhost"

# Static files
STATIC_URL = "static/" # URL prefix for static files
STATICFILES_DIRS = [BASE_DIR / "static"] # where to find them in development
STATIC_ROOT = BASE_DIR / "staticfiles" # where collectstatic puts them for production

# After login, go to dashboard
LOGIN_REDIRECT_URL = "dashboard" # named URL
LOGOUT_REDIRECT_URL = "home"
LOGIN_URL = "login" # @login_required sends here
```

CHAPTER 07

Database Models

Defining Tables, Fields, and Relationships

CHAPTER 07

Database Models — Storing Data

A Model is a Python class that represents a database table. Each attribute of the class represents a column in the table. Django automatically generates SQL CREATE TABLE statements from your model definitions.

How Models Become Database Tables

```
# models.py – Python class definition
class Service(models.Model):
    name = models.CharField(max_length=100)
    price = models.DecimalField(max_digits=8, decimal_places=2)
    is_active = models.BooleanField(default=True)

# Django generates this SQL automatically:
# CREATE TABLE garage_service (
#     id          INTEGER PRIMARY KEY AUTOINCREMENT,
#     name        VARCHAR(100) NOT NULL,
#     price        DECIMAL(8,2) NOT NULL,
#     is_active    BOOLEAN NOT NULL DEFAULT 1
# );

# Using the model in Python:
Service.objects.create(name="Oil Change", price=1200.00)
# SQL: INSERT INTO garage_service (name, price, is_active) VALUES ('Oil Change', 1200.00, 1)

service = Service.objects.get(name="Oil Change")
# SQL: SELECT * FROM garage_service WHERE name = 'Oil Change' LIMIT 1

services = Service.objects.filter(is_active=True)
# SQL: SELECT * FROM garage_service WHERE is_active = 1

services = Service.objects.filter(is_active=True).order_by("name")
# SQL: SELECT * FROM garage_service WHERE is_active = 1 ORDER BY name ASC
```

Field Types — What Each One Does

CharField(max_length=100)

Short text up to 100 characters. e.g. name, vehicle number

TextField()	Long text with no length limit. e.g. notes, description
DecimalField(max_digits=8, decimal_places=2)	Exact decimal number. Always use for money, never FloatField
BooleanField(default=False)	True or False. e.g. is_active, is_paid, payment_pending_verification
DateField()	A date: 2026-10-15. e.g. appointment_date
TimeField()	A time: 14:30. e.g. appointment_time
DateTimeField(auto_now_add=True)	Timestamp — automatically set to current time when record created
DateTimeField(null=True)	Timestamp that can be empty — e.g. paid_at (NULL until payment)
SlugField(unique=True)	URL-safe text like 'oil-change'. unique=True means no duplicates
PositiveSmallIntegerField()	Small positive integer — e.g. rating (1-5), sort_order
EmailField()	Text field that validates it looks like an email
ForeignKey(Service, ...)	Links to another table — many bookings can reference one service
OneToOneField(User, ...)	Unique link — one user is manager of one location, and vice versa

ForeignKey — The Most Important Relationship

```
# Booking has a ForeignKey to Service
# Many bookings can reference the same service
class Booking(models.Model):
    service = models.ForeignKey(
        Service,                # points to the Service model
        on_delete=models.PROTECT, # prevent deleting a Service that has bookings
        related_name="bookings"  # Service.bookings.all() returns all bookings for it
    )
    customer = models.ForeignKey(
        settings.AUTH_USER_MODEL, # points to Django's User model
        on_delete=models.CASCADE,  # if user deleted, delete all their bookings too
        related_name="bookings"   # user.bookings.all() returns all user's bookings
    )

# In Python:
booking = Booking.objects.get(pk=42)
print(booking.service.name)          # "Oil Change" — accesses related Service row
print(booking.customer.username)    # "sujan123" — accesses related User row

user = User.objects.get(username="sujan123")
user.bookings.all()                  # all bookings by this user
user.bookings.filter(is_paid=True)  # only paid bookings

# The Booking model also has:
location = models.ForeignKey("GarageLocation", on_delete=models.PROTECT,
                             null=True, blank=True)

# null=True = database column can be NULL (empty)
# blank=True = form validation accepts empty value
```

Migrations — Keeping the Database in Sync

When you add or change a field in your model, you need to update the database. Migrations are the version control system for your database schema:

```
# Step 1: Add a new field to a model
class Booking(models.Model):
    payment_pending_verification = models.BooleanField(default=False) # NEW FIELD

# Step 2: Tell Django to generate a migration file
python manage.py makemigrations
# Creates: garage/migrations/0009_booking_payment_pending_verification.py

# Step 3: Apply the migration to actually change the database
python manage.py migrate
# SQL: ALTER TABLE garage_booking ADD COLUMN payment_pending_verification BOOLEAN DEFAULT 0

# Step 4: Never edit migration files manually – let Django manage them
# Always commit migration files to Git alongside your model changes

# Check current migration status
python manage.py showmigrations
# garage
# [X] 0001_initial
# [X] 0002_seed_catalog
# ...
# [X] 0009_booking_payment_pending_verification
```

CHAPTER 08

Forms

Accepting and Validating User Input

CHAPTER 08

Forms — Accepting User Input

```
# forms.py – BookingForm

from django import forms
from .models import Booking, Service, GarageLocation

class BookingForm(forms.ModelForm):
    # ModelForm = automatically creates fields from the model

    # Add extra fields or override defaults
    appointment_date = forms.DateField(
        widget=forms.DateInput(attrs={"type": "date"}) # renders as HTML date picker
    )

    class Meta:
        model = Booking
        fields = ("service", "location", "vehicle_type", "car_type",
                  "vehicle_number", "vehicle", "appointment_date",
                  "appointment_time", "notes")
        # Only these fields appear in the form
        # Other fields (customer, is_paid, etc.) are set in the view

        labels = {
            "vehicle": "Vehicle details (year, make & model)",
        }
        widgets = {
            "notes": forms.Textarea(attrs={"rows": 4}),
        }

    # Field-level validation: clean_<fieldname>()
    def clean_vehicle_number(self):
        value = self.cleaned_data["vehicle_number"]
        return value.strip().upper()
        # "ka 01 ab 1234" becomes "KA 01 AB 1234"

    # Cross-field validation: clean()
    def clean(self):
        cleaned = super().clean()
        vehicle_type = cleaned.get("vehicle_type")
```

```

car_type      = cleaned.get("car_type")

if vehicle_type == "car" and not car_type:
    self.add_error("car_type", "Choose your car type.")
    # attaches error to the car_type field specifically
return cleaned

```

Using a Form in a View

```

# views.py – book_service view

@login_required
def book_service(request):
    # GET request: show empty form
    # POST request: process submitted form

    initial = {"service": request.GET.get("service")}
    # If URL is /book/?service=3, pre-select service with id=3

    form = BookingForm(request.POST or None, initial=initial)
    # request.POST or None:
    #   On GET → request.POST is empty dict → form = BookingForm(None) = empty form
    #   On POST → request.POST has data → form = BookingForm(data) = bound form

    if request.method == "POST" and form.is_valid():
        # is_valid() runs all clean_ methods and validates each field

        booking = form.save(commit=False)
        # commit=False = create Python object but DON'T save to database yet

        booking.customer = request.user
        # Attach the logged-in user – customer field can't come from form (security)

        booking.save()
        # NOW save to database (triggers model's clean() method too)

        messages.success(request, "Your appointment request is in!")
        return redirect("booking_history")
        # POST → Redirect → GET pattern: prevents double-submission on refresh

    return render(request, "garage/booking_form.html", {"form": form})
    # On validation failure: re-renders form WITH error messages displayed

```

Rendering Forms in Templates

```

<!-- booking_form.html -->
<form method="post" action="{% url 'book_service' %}">
    {% csrf_token %}
    <!-- CSRF token = hidden field with security token to prevent cross-site attacks -->

    <!-- Render each field manually for full design control -->
    <div class="form-field">
        <label for="{{ form.service.id_for_label }}">
            {{ form.service.label }}
        </label>
        {{ form.service }} <!-- renders as <select> dropdown -->
        {% if form.service.errors %}
            <span class="field-error">{{ form.service.errors.0 }}</span>
        {% endif %}
    </div>

    <div class="form-field">

```

```
<label>{{ form.appointment_date.label }}</label>
{{ form.appointment_date }} <!-- renders as <input type="date"> -->
{% for error in form.appointment_date.errors %}
    <span class="field-error">{{ error }}</span>
{% endfor %}
</div>

<button type="submit" class="button button-red">Book Service</button>
</form>
```

CHAPTER 09

Views

The Business Logic — Handling Requests

CHAPTER 09

Views — The Business Logic

Every view function takes a request and returns a response. Here are the most important patterns:

Pattern 1 — Simple Page Render

```
def services(request):
    services = Service.objects.filter(is_active=True)
    return render(request, "garage/services.html", {"services": services})
# render() = load template, fill variables, return HttpResponse with HTML
```

Pattern 2 — Login Required + Database Query

```
@login_required # redirect to login if not authenticated
def dashboard(request):
    # request.user = the currently logged-in User object
    bookings = request.user.bookings.select_related("service", "location").all()
    # .bookings = all bookings for this user (from related_name="bookings")
    # .select_related() = SQL JOIN to fetch service + location in one query (no N+1)
    # .all() = fetch all rows

    upcoming = [b for b in bookings if b.is_upcoming][:3]
    # list comprehension = filter using Python property, take first 3

    return render(request, "garage/dashboard.html", {
        "upcoming": upcoming,
        "booking_count": bookings.count(),
    })
```

Pattern 3 — Form Processing (GET + POST)

```
def contact(request):
    form = ContactForm(request.POST or None)
    # GET: request.POST is {} (empty) → request.POST or None = None → blank form
    # POST: request.POST has data → bound form ready for validation

    if request.method == "POST" and form.is_valid():
```

```

        form.save() # saves ContactMessage to database
        messages.success(request, "Message received!")
        return redirect("contact") # PRG: Post → Redirect → Get

    return render(request, "garage/contact.html", {"form": form})

```

Pattern 4 — URL Parameter + Security Scoping

```

@login_required
def invoice_detail(request, booking_id):
    # booking_id comes from URL: /account/bookings/42/invoice/

    booking = get_object_or_404(
        request.user.bookings.select_related("service", "location"),
        pk=booking_id
    )
    # SECURITY: request.user.bookings restricts to THIS user's bookings
    # If user 7 tries /account/bookings/99/invoice/ and booking 99 belongs to user 8:
    # → get_object_or_404 raises Http404 (not found for this user)
    # User 7 can NEVER access user 8's data

    if not booking.has_invoice:
        raise Http404("Invoice not ready yet.")

    return render(request, "garage/invoice.html", {"booking": booking})

```

Pattern 5 — POST-Only Action with State Change

```

@login_required
def pay_booking_upi(request, booking_id):
    if request.method != "POST":
        return HttpResponse(status=405) # 405 Method Not Allowed

    booking = get_object_or_404(request.user.bookings, pk=booking_id)

    if booking.is_paid:
        messages.info(request, "Already marked as paid.")
        return redirect("invoice_detail", booking_id=booking.pk)

    form = UpiPaymentConfirmationForm(request.POST)
    if form.is_valid():
        ref = form.cleaned_data.get("payment_reference", "").strip()

        # State change: mark as pending verification
        booking.payment_pending_verification = True
        booking.is_paid = False # admin must verify
        booking.is_ready_for_delivery = False
        booking.payment_reference = ref or booking.payment_reference
        booking.save(update_fields=[
            "payment_pending_verification",
            "payment_reference",
            "is_paid",
            "is_ready_for_delivery"
        ])
    # update_fields = only update these columns (efficient – not all 20 fields)

    messages.success(request, f"UTR '{ref}' submitted! Admin will verify soon.")
    return redirect("invoice_detail", booking_id=booking.pk)

```

Flash Messages — Django's Notification System

```
# In a view:
from django.contrib import messages

messages.success(request, "Booking confirmed!") # green
messages.error(request, "Payment failed.")      # red
messages.info(request, "Already paid.")         # blue
messages.warning(request, "Booking is pending.") # yellow

# In base.html template:
{% for message in messages %}
    <div class="message message-{{ message.tags }}">
        {{ message }}
    </div>
{% endfor %}

# Messages are stored in the session and shown ONCE on the next page load
# After being displayed, they are removed automatically
```

CHAPTER 10

Templates

Django Template Language — The HTML Layer

CHAPTER 10

Templates — The HTML Layer

Django Template Language (DTL) — Complete Reference

```

<!-- 1. Variables – display data from the view -->
{{ booking.customer.username }}      → "sujan123"
{{ service.price }}                  → 2500.00
{{ booking.effective_price }}        → accesses @property, same syntax as field

<!-- 2. Filters – transform variable values -->
{{ service.price|floatformat:0 }}     → "2500" (0 decimal places)
{{ text|truncatechars:45 }}          → "Oil change and full service inclu..."
{{ review.customer_name|first|upper }} → "S" (first letter, uppercased)
{{ locations|length }}               → 3 (count of items in list)
{{ appointment_date|date:"d M Y" }}  → "15 Oct 2026"

<!-- 3. Tags – control flow and logic -->
{% if user.is_authenticated %}
    <a href="{% url 'logout' %}">Logout</a>
{% elif user.is_staff %}
    <a href="{% url 'management_dashboard' %}">Admin</a>
{% else %}
    <a href="{% url 'login' %}">Login</a>
{% endif %}

<!-- 4. For loops -->
{% for service in services %}
    <article class="service-card">
        <h3>{{ service.name }}</h3>
        <p>■{{ service.price|floatformat:0 }}</p>
    </article>
{% empty %}
    <p>No services available.</p> <!-- shown when services list is empty -->
{% endfor %}

<!-- 5. URL generation – never hardcode URLs -->
<a href="{% url 'book_service' %}">Book</a>
<a href="{% url 'invoice_detail' booking.id %}">View Invoice</a>

```

```

<a href="{% url 'pay_booking_upi' booking.pk %}">Pay</a>

<!-- 6. Static files -->
<link rel="stylesheet" href="{% static 'css/site.css' %}">


<!-- 7. Template inheritance -->
{% extends 'base.html' %}      <!-- inherit from base template -->
{% block title %}Dashboard{% endblock %}
{% block content %}
    <!-- page-specific content here -->
{% endblock %}

```

base.html — The Master Layout

```

<!-- templates/base.html – every page inherits this -->
<!DOCTYPE html>
<html lang="en">
<head>
    <title>{% block title %}THE GARAGE{% endblock %}</title>
    <!-- Child templates override: {% block title %}Dashboard{% endblock %} -->
</head>
<body>
    <nav class="main-nav">
        <a href="{% url 'home' %}">Home</a>
        <a href="{% url 'services' %}">Services</a>

        {% if user.is_authenticated %}
            <a href="{% url 'dashboard' %}">My Account</a>
            <a href="{% url 'management_login' %}" class="nav-admin-link">Admin</a>
            <form method="post" action="{% url 'logout' %}">
                {% csrf_token %}
                <button type="submit">Logout</button>
            </form>
        {% else %}
            <a href="{% url 'login' %}">Login</a>
            <a href="{% url 'signup' %}" class="button nav-cta">Book Now</a>
        {% endif %}
    </nav>

    <!-- Flash messages from views -->
    {% if messages %}
    <div class="message-stack">
        {% for message in messages %}
            <div class="message">{{ message }}</div>
        {% endfor %}
    </div>
    {% endif %}

    <main>
        {% block content %}{% endblock %}
        <!-- Each child template fills this block -->
    </main>

    <footer>...</footer>
    <script src="{% static 'js/site.js' %}"></script>
</body>
</html>

```

CHAPTER 11

Authentication

Login System — Two Types of Users

CHAPTER 11

Authentication — The Login System

Django comes with a complete, production-ready authentication system built in. You get user accounts, password hashing, login/logout, and session management for free. No passwords are ever stored in plain text — Django uses PBKDF2 hashing by default.

How Django Authentication Works

```
# Django's built-in User model has these fields:
# id, username, password (hashed), email, first_name, last_name,
# is_active, is_staff, is_superuser, date_joined, last_login

# The is_staff flag is key:
# is_staff = False → regular customer (default for new signups)
# is_staff = True  → admin / branch manager (set manually in /admin/)

# In views:
request.user          # the currently logged-in User object
request.user.is_authenticated # True if logged in, False if not
request.user.username # "sujan123"
request.user.is_staff  # True/False – is this an admin?
request.user.get_full_name() # "Sujan CR"

# Login programmatically
from django.contrib.auth import login
login(request, user) # creates a session for this user

# Check password and log in
from django.contrib.auth import authenticate
user = authenticate(request, username=username, password=password)
if user is not None:
    login(request, user)
```

Two Login Types in THE GARAGE

```
# LOGIN TYPE 1: Customer Login
# URL: /accounts/login/
```

```
# Django's built-in LoginView – no custom code needed
# Any user can log in here
# After login → settings.LOGIN_REDIRECT_URL = "dashboard" → /account/

# In urls.py:
path("accounts/login/", auth_views.LoginView.as_view(
    template_name="registration/login.html"
), name="login"),

# LOGIN TYPE 2: Admin/Staff Login
# URL: /management/login/
# Custom view – only allows is_staff=True users

def management_login(request):
    form = AuthenticationForm(request=request, data=request.POST or None)
    if request.method == "POST" and form.is_valid():
        user = form.get_user()

        if not user.is_staff:
            form.add_error(None, "This account does not have administrator access.")
            # form.add_error(None, ...) = global error, not attached to a field
        else:
            login(request, user)
            # Decide where to send the admin after login
            if GarageLocation.objects.filter(manager=user, is_active=True).exists():
                return redirect("branch_dashboard") # branch manager
            return redirect("management_dashboard") # central admin

    return render(request, "garage/management_login.html", {"form": form})
```

The User Signup Flow

```
# forms.py – extending Django's built-in UserCreationForm
class SignUpForm(UserCreationForm):
    email = forms.EmailField()
    first_name = forms.CharField(max_length=150)

    class Meta:
        model = User
        fields = ("first_name", "last_name", "username", "email",
            "password1", "password2")
        # password1 + password2 = enter password twice to confirm
        # Django validates they match and hashes the final password

# views.py – signup
def signup(request):
    form = SignUpForm(request.POST or None)
    if request.method == "POST" and form.is_valid():
        user = form.save() # creates User with hashed password
        login(request, user) # log them in immediately after signup
        messages.success(request, "Welcome! Your account is ready.")
        return redirect("dashboard")
    return render(request, "registration/signup.html", {"form": form})
```

CHAPTER 12

The UPI Payment Flow

From Invoice to Admin Verification to Delivery Notice

CHAPTER 12

The UPI Payment Flow

UPI (Unified Payments Interface) is India's real-time payment system. Unlike credit cards, you don't need a payment gateway — you use a deep-link URL that opens the user's UPI app (Google Pay, PhonePe, Paytm) with the amount pre-filled.

The Booking Payment States

State 1: Invoice Ready	is_paid=False, payment_pending_verification=False → Customer sees the invoice with UPI QR code and UTR input form
State 2: UTR Submitted	is_paid=False, payment_pending_verification=True → Customer sees 'Awaiting admin verification' → Admin sees '■■ UPI Verification Needed' with the UTR number
State 3: Admin Verified	is_paid=True, is_ready_for_delivery=True, payment_pending_verification=False → Customer sees '■ Vehicle is READY FOR DELIVERY!' banner

The UPI Deep-Link URL

```
# models.py – Booking model
@property
def upi_payment_url(self):
    amount_str = f"{self.effective_price:.2f}"
    return (
        f"upi://pay?"
        f"pa=8431699047@nyes"           # pa = payee address (UPI ID)
        f"&pn=SUJAN%20C%20R"         # pn = payee name (URL-encoded spaces = %20)
        f"&am={amount_str}"           # am = amount (e.g. 2500.00)
        f"&cu=INR"                     # cu = currency
        f"&tn=Invoice%20TG-{self.pk:06d}%20THE%20GARAGE" # tn = transaction note
    )

# Example result:
# upi://pay?pa=8431699047@nyes&pn=SUJAN%20C%20R&am=2500.00&cu=INR&tn=Invoice%20TG-000042

# In invoice.html:
```

```
<a href="{{ booking.upi_payment_url }}" class="button button-upi">
    Pay ■{{ invoice_total }} via UPI ■
</a>
# On mobile: clicking this opens Google Pay / PhonePe / Paytm automatically
# with amount ■2500 and payee pre-filled
```

Complete Flow — Code at Each Step

```
# STEP 1: Customer opens invoice page
# views.py → invoice_detail
booking = get_object_or_404(request.user.bookings, pk=booking_id)
payment_form = UpiPaymentConfirmationForm()
return render(request, "garage/invoice.html", {
    "booking": booking,
    "payment_form": payment_form,
})

# STEP 2: invoice.html shows the form
# <form method="post" action="{% url 'pay_booking_upi' booking.id %}">
#     {% csrf_token %}
#     {{ payment_form.payment_reference }} ← UTR input field
#     <button type="submit">Confirm Payment Submitted</button>
# </form>

# STEP 3: Customer submits form → pay_booking_upi view
booking.payment_pending_verification = True
booking.is_paid = False # explicitly False
booking.is_ready_for_delivery = False
booking.payment_reference = "UTR123456789012" # what customer entered
booking.save(update_fields=[...])

# STEP 4: Admin logs into /management/ and sees:
# {% if booking.payment_pending_verification and not booking.is_paid %}
# <form method="post" action="{% url 'management_booking_action' booking.id %}">
#     <input type="hidden" name="action" value="verify_payment">
#     UTR on file: {{ booking.payment_reference }}
#     <button>Verify & Clear Delivery ✓</button>
# </form>
# {% endif %}

# STEP 5: Admin clicks verify → management_booking_action view
elif action == BranchBookingActionForm.Action.VERIFY_PAYMENT:
    booking.is_paid = True
    booking.is_ready_for_delivery = True
    booking.paid_at = timezone.now()
    booking.payment_pending_verification = False
    booking.save(update_fields=[
        "is_paid", "is_ready_for_delivery",
        "paid_at", "payment_pending_verification"
    ])

# STEP 6: Customer refreshes page – delivery notice appears
# {% if booking.is_vehicle_ready %}
# <div class="delivery-ready-banner">
#     ■ Your vehicle is READY FOR DELIVERY!
# </div>
# {% elif booking.payment_pending_verification %}
# <div class="verification-notice-strip">
#     ■ Awaiting admin verification – UTR: {{ booking.payment_reference }}
# </div>
# {% endif %}

# booking.is_vehicle_ready is a @property:
```

```
# @property
# def is_vehicle_ready(self):
#     return self.is_paid and self.is_ready_for_delivery
```

CHAPTER 13

PDF Generation

Creating Invoice PDFs with ReportLab

CHAPTER 13

PDF Generation with ReportLab

```
# views.py – invoice_pdf

from io import BytesIO                                # in-memory file buffer
from reportlab.lib.pagesizes import A4                # A4 paper size in points
from reportlab.pdfgen import canvas                  # ReportLab drawing engine
from reportlab.lib import colors

def invoice_pdf(request, booking_id):
    booking = get_object_or_404(request.user.bookings, pk=booking_id)

    if not booking.has_invoice:
        raise Http404("Invoice not ready yet.")

    # Create in-memory buffer – no files on disk
    buffer = BytesIO()
    pdf = canvas.Canvas(buffer, pagesize=A4)

    # A4 dimensions in points (1 point = 1/72 inch)
    width, height = A4    # width=595.27, height=841.89

    # IMPORTANT: ReportLab's (0,0) is BOTTOM-LEFT corner
    # y=841 is the top of the page, y=0 is the bottom
    left = 52
    y = height - 58    # start near the top

    # Draw company name
    pdf.setFont("Helvetica-Bold", 22)
    pdf.setFillColorRGB(0.07, 0.07, 0.08)    # near black (0.0–1.0 scale, not 0–255)
    pdf.drawString(left, y, "THE GARAGE")

    # Draw invoice number – right-aligned
    pdf.setFont("Helvetica", 10)
    pdf.drawRightString(width - left, y + 5, f"INVOICE TG-{booking.pk:06d}")
    # drawRightString = text ends at the x coordinate (right-align)
    # f"{booking.pk:06d}" = "000042" (pad with leading zeros to 6 digits)

    y -= 22
```

```
# Draw a horizontal line
pdf.setStrokeColorRGB(0.82, 0.82, 0.82) # light gray
pdf.setLineWidth(0.5)
pdf.line(left, y, width - left, y)

# Payment status (green if paid, red if not)
y -= 50
if booking.is_paid:
    pdf.setFillColorRGB(0.09, 0.63, 0.36) # green
    pdf.setFont("Helvetica-Bold", 11)
    pdf.drawString(left, y, "PAYMENT STATUS: PAID IN FULL")
    pdf.drawString(left, y - 14, f"Ref: {booking.payment_reference}")
else:
    pdf.setFillColorRGB(0.85, 0.25, 0.20) # red
    pdf.drawString(left, y, "PAYMENT STATUS: DUE")

# Finalize and return
pdf.save() # writes all drawing commands to the buffer

response = HttpResponse(
    buffer.getvalue(), # raw PDF bytes
    content_type="application/pdf"
)
response["Content-Disposition"] = f'attachment; filename="garage-invoice-{booking.pk}.pdf"'
# Content-Disposition: attachment → browser downloads file instead of displaying
return response
```

CHAPTER 14

Testing

Writing Automated Tests — All 23 Tests Explained

CHAPTER 14

Testing — Making Sure It Works

Automated tests are programs that test your program. They run every single important scenario automatically so you know if a code change broke something. This project has **23 tests** — all must pass before deploying.

How Django Tests Work

```
# garage/tests.py

from django.test import TestCase
from django.urls import reverse
from django.contrib.auth.models import User
from .models import Booking, Service, GarageLocation

class GarageWorkflowTests(TestCase):
    # TestCase creates a fresh empty test database before each test
    # and throws it away after – tests never affect each other

    def setUp(self):
        # setUp() runs before EVERY individual test
        # Create common test data here

        self.customer = User.objects.create_user(
            username="testcustomer",
            password="testpass123"
        )
        self.staff = User.objects.create_user(
            username="teststaff",
            password="testpass123",
            is_staff=True # admin flag
        )
        self.service = Service.objects.create(
            name="Oil Change",
            slug="oil-change",
            summary="Full synthetic oil change",
            description="...",
            price=1200.00
        )
```

```

self.location = GarageLocation.objects.create(
    name="Bangalore Central",
    address="MG Road",
    city="Bangalore",
    region="KA",
    postal_code="560001",
    latitude=12.9716,
    longitude=77.5946
)

def test_home_and_service_catalog_render(self):
    # self.client is a fake browser
    response = self.client.get("/")          # GET homepage
    self.assertEqual(response.status_code, 200) # must return 200 OK

    response = self.client.get(reverse("services"))
    self.assertEqual(response.status_code, 200)
    # reverse("services") converts URL name to path: "/services/"

def test_booking_requires_login(self):
    response = self.client.get(reverse("book_service"))
    # Not logged in → should redirect to login page
    self.assertRedirects(response, "/accounts/login/?next=/book/")

def test_customer_can_create_future_booking(self):
    self.client.force_login(self.customer) # log in without password

    from datetime import date, timedelta
    future_date = (date.today() + timedelta(days=3)).isoformat()

    response = self.client.post(reverse("book_service"), {
        "service":          self.service.pk,
        "location":         self.location.pk,
        "vehicle_type":     "car",
        "car_type":         "sedan",
        "vehicle_number":   "KA01AB1234",
        "vehicle":          "2022 Honda City",
        "appointment_date": future_date,
        "appointment_time": "10:00",
        "notes":            "Please check AC too",
    })
    self.assertRedirects(response, reverse("booking_history"))
    self.assertEqual(Booking.objects.count(), 1) # one booking was created

```

The UPI + Admin Verification Test

```

def test_customer_can_pay_via_upi_and_see_vehicle_ready_for_delivery(self):
    # Create a completed booking with invoice ready
    from datetime import date, timedelta
    booking = Booking.objects.create(
        customer=self.customer,
        service=self.service,
        location=self.location,
        vehicle_number="KA01AB1234",
        vehicle="2022 Honda City",
        vehicle_type="car",
        car_type="sedan",
        appointment_date=date.today() + timedelta(days=1),
        appointment_time="10:00",
        status="completed",
        final_price=1200.00,
        invoice_ready=True,
    )

```

```
)

# 1. Customer submits UTR number
self.client.force_login(self.customer)
response = self.client.post(
    reverse("pay_booking_upi", args=[booking.pk]),
    {"payment_reference": "UTR123456789012"}
)

# 2. Check state: pending verification, NOT paid yet
booking.refresh_from_db() # re-read fields from database
self.assertTrue(booking.payment_pending_verification)
self.assertFalse(booking.is_paid) # admin hasn't verified yet
self.assertFalse(booking.is_ready_for_delivery)

# 3. Admin verifies the payment
self.client.force_login(self.staff)
self.client.post(
    reverse("management_booking_action", args=[booking.pk]),
    {"action": "verify_payment"}
)

# 4. Check final state: paid, ready for delivery
booking.refresh_from_db()
self.assertTrue(booking.is_paid)
self.assertTrue(booking.is_ready_for_delivery)
self.assertFalse(booking.payment_pending_verification)
self.assertTrue(booking.is_vehicle_ready) # @property should now be True
```

Run Tests

```
# Run all 23 tests
.venv\Scripts\python.exe manage.py test

# Run with verbose output (shows each test name)
.venv\Scripts\python.exe manage.py test --verbosity=2

# Run a specific test
.venv\Scripts\python.exe manage.py test garage.tests.GarageWorkflowTests.test_home_and_service_catalog_render

# Expected output when all pass:
# Ran 23 tests in 12.215s
# OK
```

CHAPTER 15

Docker

Containerization — Run Anywhere, Same Way

CHAPTER 15

Docker — Containerization

A Docker **container** is like a box that contains your application AND everything it needs to run — Python, all libraries, all settings. The box runs identically on your laptop, on a server in Mumbai, or on a server in Singapore. 'It works on my machine' is no longer a problem.

Key Docker Concepts

Image	A blueprint/template for containers. Built from a Dockerfile. Like a class in Python.
Container	A running instance of an image. Like an object (instance) of a class.
Dockerfile	Instructions for building an image. Layer by layer.
DockerHub	Cloud registry for images. Like GitHub but for Docker images. Free for public images.
docker build	Read Dockerfile and create an image
docker run	Start a container from an image
docker push	Upload image to DockerHub
docker pull	Download image from DockerHub
Layer cache	Each Dockerfile instruction is a layer. If layer unchanged, Docker reuses it (fast rebuilds)
Port mapping	-p 8000:8000 = your port 8000 connects to container's port 8000

Dockerfile — Line by Line

```
FROM python:3.12-slim
# Start from official Python 3.12 image on Debian slim (~150MB)
# 'slim' = minimal Debian, no extra packages (smaller image)

ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    DJANGO_SETTINGS_MODULE=the_garage.settings \
```

```
    PORT=8000
# Set environment variables inside the container:
# PYTHONDONTWRITEBYTECODE=1 → don't create .pyc files (saves space)
# PYTHONUNBUFFERED=1 → print() appears in logs immediately (not buffered)

WORKDIR /app
# All subsequent commands run from /app directory inside the container

RUN apt-get update && apt-get install -y --no-install-recommends \
    build-essential libjpeg-dev zlib1g-dev curl \
    && rm -rf /var/lib/apt/lists/*
# Install system libraries needed by reportlab and Pillow (image processing)
# --no-install-recommends = don't install optional packages (smaller image)
# rm -rf /var/lib/apt/lists/* = delete apt cache → smaller layer

COPY requirements.txt /app/
# Copy ONLY requirements.txt first (not the whole project)

RUN pip install --no-cache-dir -r requirements.txt
# Install Python packages
# --no-cache-dir = don't save download cache (smaller image)
# LAYER CACHING: if requirements.txt hasn't changed since last build,
# Docker reuses this layer – very fast rebuild

COPY . /app/
# NOW copy all source code
# If only source code changed (not requirements), only this layer rebuilds

RUN python manage.py collectstatic --noinput
# Collect all CSS/JS/images into /app/staticfiles/
# This happens at BUILD time, not runtime
# Nginx will serve these static files directly

RUN useradd -m -u 1000 appuser && chown -R appuser:appuser /app
USER appuser
# SECURITY: never run as root in production
# Create a regular user, give them ownership of /app, switch to that user
# If an attacker exploits the app, they can't become root

EXPOSE 8000
# Document that container uses port 8000 (informational only)

HEALTHCHECK --interval=30s --timeout=5s --start-period=10s --retries=3 \
    CMD curl -f http://localhost:8000/ || exit 1
# Docker checks this every 30 seconds
# If curl fails 3 times → container marked "unhealthy"
# --start-period=10s = wait 10 seconds before first check (let app start)

ENTRYPOINT ["/app/docker-entrypoint.sh"]
CMD ["gunicorn", "the_garage.wsgi:application", "--bind", "0.0.0.0:8000", "--workers", "3"]
# ENTRYPOINT = always runs first (runs migrations via docker-entrypoint.sh)
# CMD = default command (starts Gunicorn web server)
# --workers 3 = 3 parallel worker processes (rule: 2 * CPU_cores + 1)
```

CHAPTER 16

Jenkins CI/CD Pipeline

Automated Testing, Building, and Deployment

CHAPTER 16

Jenkins — CI/CD Pipeline

CI/CD stands for Continuous Integration and Continuous Delivery. Every time you push code to Git, Jenkins automatically: runs all 23 tests, builds a Docker image, tests the container, pushes it to DockerHub, and deploys it to the production server. No manual steps. If a test fails, the deployment never happens.

All 8 Pipeline Stages

```
pipeline {
    agent any    // run on any available Jenkins agent (worker machine)

    environment {
        DOCKER_IMAGE_NAME = 'mastertoinfinity/the-garage'
        DOCKER_TAG         = "${env.BUILD_NUMBER}" // auto-increments: 1, 2, 3...
        DJANGO_SECRET_KEY = credentials('django-secret-key')
        // credentials() reads from Jenkins secrets store – never in source code
    }

    options {
        buildDiscarder(logRotator(numToKeepStr: '15')) // keep only last 15 builds
        disableConcurrentBuilds() // only one pipeline at a time
        timeout(time: 30, unit: 'MINUTES') // kill if stuck > 30 minutes
    }

    stages {
        stage('Checkout') {
            steps {
                checkout scm // pull latest code from Git
            }
        }

        stage('Code Lint') {
            steps {
                sh 'python3 -m py_compile $(find . -name "*.py" -not -path "./.venv/*")'
                // Check every Python file for syntax errors
                // sh '' = run shell command on the Jenkins server
            }
        }
    }
}
```

```

stage('Run Django Test Suite') {
  steps {
    sh '''
      python3 -m venv .ci_venv          // create fresh virtual env
      . .ci_venv/bin/activate           // activate it
      pip install -r requirements.txt    // install dependencies
      python manage.py test             // run ALL 23 tests
      deactivate
    '''
    // If ANY test fails, this stage fails → pipeline STOPS → no deployment
  }
}

stage('Build Docker Image') {
  steps {
    sh '''
      docker build --tag ${DOCKER_IMAGE_NAME}:${DOCKER_TAG} .
      docker build --tag ${DOCKER_IMAGE_NAME}:latest .
    '''
    // Two tags: :42 (for rollback) and :latest (for convenience)
  }
}

stage('Docker Smoke Test') {
  steps {
    sh '''
      docker run -d --name garage_smoke_test -p 8009:8000 \
        -e DJANGO_SECRET_KEY="ci-test-key" \
        -e DJANGO_DEBUG="True" \
        ${DOCKER_IMAGE_NAME}:${DOCKER_TAG}
      sleep 6
      curl -f http://127.0.0.1:8009/ || (docker logs garage_smoke_test && exit 1)
      docker rm -f garage_smoke_test
    '''
    // Start container, wait 6s, check homepage responds, clean up
  }
}

stage('Push to DockerHub') {
  steps {
    withCredentials([usernamePassword(credentialsId: "dockerhub-credentials",
      usernameVariable: 'DOCKER_USER', passwordVariable: 'DOCKER_PASS')]) {
      sh '''
        echo "$DOCKER_PASS" | docker login -u "$DOCKER_USER" --password-stdin
        docker push ${DOCKER_IMAGE_NAME}:${DOCKER_TAG}
        docker push ${DOCKER_IMAGE_NAME}:latest
        docker logout
      '''
    }
    // withCredentials injects secrets into environment for this block only
    // --password-stdin = more secure (password not visible in process list)
  }
}

stage('Terraform Validate') {
  steps {
    dir('terraform') {
      sh 'terraform init -backend=false && terraform validate'
      // Checks Terraform syntax – no actual AWS API calls
    }
  }
}

```

```
stage('Deploy via Ansible') {
    steps {
        dir('ansible') {
            sh 'ansible-playbook -i inventory.ini playbook.yml --extra-vars "app_image=${DOCKER_IMAGE_NAME}"'
            // SSH into production server and deploy new Docker image
        }
    }
}

post {
    always {
        sh 'rm -rf .ci_venv || true'
        sh 'docker rm -f garage_smoke_test 2>/dev/null || true'
        cleanWs() // delete workspace after build
    }
    failure {
        echo "Pipeline FAILED – check logs above"
    }
}
}
```

CHAPTER 17

Terraform

Cloud Infrastructure as Code — AWS Setup

CHAPTER 17

Terraform — Cloud Infrastructure

Terraform lets you define cloud infrastructure (servers, networks, firewalls) as code in `.tf` files. Instead of clicking around the AWS console, you write what you want, run **terraform apply**, and AWS creates it. You can version control your infrastructure the same way you version control your code.

What Terraform Creates for THE GARAGE

VPC (Virtual Private Cloud)	An isolated private network in AWS. Your server lives inside it. Like your own section of the internet.
Subnet	A subdivision of the VPC. The server gets an IP address from this subnet's range.
Internet Gateway	Connects the VPC to the public internet. Without this, your server has no internet access.
Route Table	Rules: '0.0.0.0/0 → Internet Gateway' means all traffic goes to the internet.
Security Group	Cloud firewall. Allows SSH(22), HTTP(80), HTTPS(443), Unicorn(8000). Blocks everything else.
EC2 Instance	The virtual machine. Ubuntu 22.04 LTS, t3.micro (1 vCPU, 1GB RAM), 20GB SSD.
Elastic IP	A static public IP address. Stays the same even if server is restarted.

Terraform Commands

```
# 1. Initialize – download provider plugins (AWS provider ~200MB)
terraform init

# 2. Plan – show what WOULD be created/changed (no actual changes)
terraform plan
# Output:
# + aws_instance.garage_server will be created
# + aws_security_group.garage_sg will be created
# Plan: 7 to add, 0 to change, 0 to destroy.

# 3. Apply – create the actual resources
terraform apply
```

```
# Asks: "Do you want to perform these actions? (yes/no)"
# Enter: yes

# After apply completes:
# Outputs:
# server_ip   = "13.233.45.67"
# app_url     = "http://13.233.45.67:8000"
# ssh_command = "ssh -i ~/.ssh/garage-key.pem ubuntu@13.233.45.67"

# 4. Destroy – delete all created resources (stops billing)
terraform destroy

# The variables file (terraform.tfvars):
aws_region    = "ap-south-1"
instance_type = "t3.micro"
project_name  = "the-garage"
key_pair_name = "my-aws-keypair" # must already exist in AWS
```

CHAPTER 18

Ansible

Deployment Automation — Configure Servers with Code

CHAPTER 18

Ansible — Deployment Automation

Ansible connects to servers over SSH and runs tasks on them automatically. Instead of logging into your server and running 10 commands by hand each time you deploy, Ansible runs a **playbook** — a YAML file that lists all the steps.

What the Ansible Playbook Does

```
# ansible/playbook.yml
---
- name: Deploy THE GARAGE Application
  hosts: garage_servers          # defined in inventory.ini
  become: true                  # sudo – run commands as root

  tasks:
    - name: Install Docker
      apt:
        name: [docker.io, docker-compose]
        state: present          # install if not present
        update_cache: yes

    - name: Pull latest Docker image
      community.docker.docker_image:
        name: "{{ app_image }}" # e.g. mastertoinfinity/the-garage:42
        source: pull

    - name: Stop old container
      community.docker.docker_container:
        name: the-garage
        state: absent           # stop and remove

    - name: Start new container
      community.docker.docker_container:
        name: the-garage
        image: "{{ app_image }}"
        state: started
        restart_policy: always  # restart on server reboot
        ports: ["8000:8000"]
        env:
```

```

    DJANGO_SECRET_KEY: "{{ django_secret_key }}"
    DJANGO_DEBUG: "False"
    DJANGO_ALLOWED_HOSTS: "{{ server_ip }}"

- name: Copy Nginx config
  template:
    src: templates/nginx.conf.j2    # Jinja2 template
    dest: /etc/nginx/sites-enabled/garage.conf

- name: Reload Nginx
  service:
    name: nginx
    state: reloaded

# inventory.ini – list of servers to deploy to
[garage_servers]
13.233.45.67  ansible_user=ubuntu  ansible_ssh_private_key_file=~/.ssh/garage.pem

# Run the playbook:
ansible-playbook -i inventory.ini playbook.yml \
  --extra-vars "app_image=mastertoinfinity/the-garage:42 django_secret_key=abc123"

```

Why Nginx Sits in Front of Gunicorn

```

# Without Nginx:
Browser ■■■HTTP port 8000■■■■ Gunicorn (Django) – BAD
# Gunicorn is not designed for direct internet exposure
# Cannot handle SSL, static files efficiently, or many concurrent connections

# With Nginx (correct setup):
Browser ■■■HTTP port 80■■■■ Nginx ■■■proxy_pass port 8000■■■■ Gunicorn (internal only)

# Nginx responsibilities:
# ✓ SSL/HTTPS termination (handles certificate, decrypts traffic)
# ✓ Serves /static/ files directly (CSS, JS, images – bypasses Django completely)
# ✓ Connection keep-alive pooling
# ✓ Rate limiting and DDoS protection
# ✓ Gzip compression

# nginx.conf.j2 (Jinja2 template):
server {
    listen 80;
    server_name {{ server_ip }};    # {{ }} = Ansible fills this in

    location /static/ {
        alias /app/staticfiles/;    # serve files directly, no Python involved
        expires 30d;                # browser caches for 30 days
    }

    location / {
        proxy_pass http://127.0.0.1:8000;    # forward to Gunicorn
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;    # pass real client IP
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }
}

```

CHAPTER 19

Industry Standards

Best Practices Applied Throughout This Project

CHAPTER 19

Industry Standards & Best Practices

Environment variables for secrets

Secret key, database password, API keys — never hardcoded in source code. Read with `os.getenv()`. Stored in server environment or Jenkins credentials vault.

Never run containers as root

`useradd appuser + USER appuser` in Dockerfile. If attacker exploits app, they can't escalate to root.

Health check endpoint (`/healthz/`)

Returns JSON `{status: healthy, database: connected}`. Docker and load balancers ping this every 30s to know if app is alive.

Soft deletes (`is_active=False`)

Never DELETE records — set `is_active=False`. All queries filter(`is_active=True`). Data is never permanently lost.

Decimal fields for money

Always `DecimalField`, never `FloatField` for money. Float uses binary fractions — $\blacksquare 0.10 + \blacksquare 0.20 = \blacksquare 0.300000000000000004$ (wrong!). Decimal is exact.

`select_related()` to prevent N+1 queries

Without it: 1 query for bookings + 1 per booking for service = 51 queries for 50 bookings. With it: 1 JOIN query. 50x faster.

`update_fields=` on `save()`

`booking.save(update_fields=['is_paid', 'paid_at'])` — only updates changed columns in SQL. Safer and faster than updating all 20 fields.

Security-scoped queries

`request.user.bookings.get(pk=42)` instead of `Booking.objects.get(pk=42)`. First version fails if booking belongs to different user. Second version lets anyone access any booking.

PRG Pattern (Post-Redirect-Get)

After any form POST, redirect instead of re-rendering. Prevents browser's 'Resubmit form?' dialog on page refresh. Every POST view in this project ends with `return redirect(...)`.

CSRF tokens on all forms

`{% csrf_token %}` in every HTML form. Without it, a malicious website could submit forms on behalf of your logged-in users.

WebP images with fallback

WebP is 75-97% smaller than PNG/JPEG. Use with WebP as first choice, JPG/PNG as fallback for old browsers. `fetchpriority='high'` on LCP image.

Docker layer caching

`COPY requirements.txt before COPY .` — if requirements unchanged, pip install layer is reused. Rebuild takes 5 seconds instead of 2 minutes.

Versioned Docker images

`:42` (build number) and `:latest` tags. Can rollback by running old image: `docker run mastertoinfinity/the-garage:41`.

Automated tests before deploy

Jenkins runs all 23 tests before building image. If ANY test fails, pipeline stops — no broken code ever reaches production.

Prefers-reduced-motion media query

`@media (prefers-reduced-motion: reduce) { * { animation: none !important; } }` — disables animations for users with epilepsy or vestibular disorders.

CHAPTER 20

What to Learn Next

Your Roadmap from Here — Ordered by Priority

CHAPTER 20

What to Learn Next

Month 1 — Strengthen Fundamentals

- **Python deeply** — list comprehensions, generators, context managers, classes and inheritance. Book: 'Automate the Boring Stuff with Python' (free online)
- **Django ORM** — practice `annotate()`, `aggregate()`, `Q()` objects, `F()` expressions, `transaction.atomic()`
- **Git** — branch, merge, rebase, pull request workflow. Practice on GitHub daily.
- **CSS Grid and Flexbox** — build 5 different layouts from scratch without any framework
- **PostgreSQL** — replace SQLite in this project with Postgres. Learn `psql` commands.

Month 2 — Build on the Project

- **Django REST Framework** — add API endpoints to this project. Build a simple mobile app backend.
- **Django signals** — send a real email when a booking is confirmed using `post_save` signal + Django's email backend
- **Django admin customization** — add `list_display`, `search_fields`, `list_filter`, inline models to `admin.py`
- **docker-compose** — run Postgres + Django + Nginx all together locally in one command
- **GitHub Actions** — free CI/CD alternative to Jenkins. Set up automatic tests on every push.

Month 3 — Cloud and DevOps

- **Deploy THE GARAGE to AWS** — run the Terraform files for real. Get a live URL.
- **Add SSL/HTTPS** — use Let's Encrypt + Certbot with Nginx for free HTTPS
- **AWS basics** — EC2, S3 (file storage), RDS (managed Postgres), Route 53 (DNS)
- **Monitoring** — set up Prometheus + Grafana or use AWS CloudWatch to monitor your server
- **Load testing** — use `locust.io` to see how many concurrent users your app handles

Month 4 — Advanced Topics

- **Redis + Celery** — background tasks (send email asynchronously, send SMS after booking confirmed)
- **WebSockets / Django Channels** — real-time updates (show delivery status without page refresh)
- **Elasticsearch** — add fast full-text search to find bookings/services
- **React or Vue.js** — learn a JavaScript framework to build interactive frontends
- **Kubernetes** — orchestrate multiple Docker containers at scale

Essential Resources

Django Docs	docs.djangoproject.com — the official documentation. Read it cover to cover.
MDN Web Docs	developer.mozilla.org — the best HTML/CSS/JS reference. Bookmark it.
Real Python	realpython.com — practical Python and Django tutorials
CSS Tricks	css-tricks.com — in-depth CSS guides and examples
Docker Docs	docs.docker.com — official Docker documentation
Terraform Registry	registry.terraform.io — browse all available Terraform providers
GitHub	github.com — create a free account, push this project, build a portfolio
Stack Overflow	stackoverflow.com — when you're stuck, search here first
ChatGPT / AI	Ask AI to explain any concept, but always read the output critically and test it

You built a complete, production-grade web application. Most developers spend years learning what this project teaches. The fact that you built it — even with AI help — means you have a codebase to study, run, break, and fix. That is the best way to learn. Read every file. Change one thing at a time. See what breaks. Fix it. Repeat.

— THE GARAGE Learning Guide · 2026 —